miLife Service Desk — Context, Plan, Tech Stack & Implementation Phases

Oct 15, 2026 · @Mitech

We are building miLife Service Desk, an internal ticketing system for miLife Insurance (Ghana), on Next.js, TypeScript, MySQL (Prisma), Tailwind and Zod, hosted on Vercel, in ten phases. Phases 0–4 deliver the usable core product; phases 5–9 add depth.

Context

Any staff member in any of the 12 departments can send a request to any department. Each department has a head who sees every request sent to it, assigns work, handles escalations and reads reports. A department can forward a ticket to another department, and the ticket keeps its full journey: every department it visited, how long it stayed and why it moved.

Departments (12): Claims, Customer Service, Underwriting, IT, HR, Finance, Compliance, Operations, Sales & Distribution, Legal, Facilities, Marketing. **Branches:** Achimota HQ, Kumasi, Takoradi.

Roles

Role	Sees	Can do
Member (staff)	Tickets they raised, tickets assigned to them, tickets where they or their department are @mentioned	Create tickets to any department, reply, rate, reopen
Head	All tickets currently with their department, plus ones that passed through; confidential ones for their department	Assign, change priority and status, forward, bulk actions, acknowledge escalations, workload, department reports, announcements
Admin	Everything across all departments	Everything, plus Administration (departments, people, SLA, rules, templates, channels, retention, audit log)

Confidentiality. HR, Legal and Compliance tickets are confidential by default. A confidential ticket is visible only to its requester, its assignee, that department's head and admins.

Deadlines (SLA). The clock starts at creation and keeps running across forwards, because it measures the requester's real wait. A ticket turns amber at 75% elapsed and is breached at 100%. A breach creates a level-1 escalation to the head, then a level-2 escalation to the COO/admin if it stays unresolved. Changing priority recomputes the due date from the creation time.

Priority	First response	Resolve within
Urgent	30 min	4 h
High	1 h	8 h
Medium	4 h	1 day
Low	8 h	3 days

What we start from. A high-fidelity HTML prototype (v7) with in-memory sample data, 17 screenshots, a README spec and a build plan. Colours, type, spacing, copy and interactions are final. The prototype is a visual reference only; we never ship or import it. Its sample data (departments, service catalogue and form fields, canned replies, templates, articles, assets and 22 sample tickets) becomes our seed.

Decisions taken

The handoff assumed PostgreSQL, Auth.js, a long-running worker and S3. Our choices change five things.

Decision	What we chose	Why	Consequence
Hosting	Vercel + managed MySQL 8	Team choice	Serverless functions: no long-running worker, no persistent disk, connection limits matter
Database	MySQL 8 instead of PostgreSQL	Team choice	`pg-boss` is out; ticket ids via auto-increment; search via MySQL FULLTEXT; filterable data in real columns, not JSON

Decision	What we chose	Why	Consequence
Background jobs	Vercel Cron calling `/api/cron/*` route handlers every minute	No worker process on Vercel	Needs Vercel Pro (every-minute crons); every job must be idempotent and fast
Authentication	Our own database sessions, email + password only	Server-side revocation; Auth.js Credentials forces JWT sessions	About 200 lines we own: argon2 hashing, httpOnly cookie, rate-limited login
Accounts	Admin creates users; system emails a set-password link	Internal tool, no public sign-up	Transactional email needed from Phase 1
Attachments	`lib/storage.ts` interface: local `./uploads` in development, Vercel Blob in production	Vercel's filesystem is temporary and read-only	Switching to R2 or S3 later means changing one file
Email	`lib/email.ts` interface: console log in development, Resend in production	Works well with Vercel	Needs a verified sending domain

Ticket ids. `Ticket.number` is an auto-increment integer with the table's counter set to start at 24827. The `MLT-` prefix is added only for display and parsed back in URLs and search. This gives sequential ids with no race conditions and no counter table.

Out of scope for launch: Microsoft sign-in, SMS, WhatsApp and Teams channels (shown as disabled toggles only), and a live policy admin integration (we read a stub table).

Tech stack

Layer	Choice	Notes
Framework	Next.js (App Router), React Server Components, Server Actions, route handlers	Reads in Server Components, writes in Server Actions; route handlers for cron, uploads and webhooks
Language	TypeScript, `strict` on	No `any` in domain code

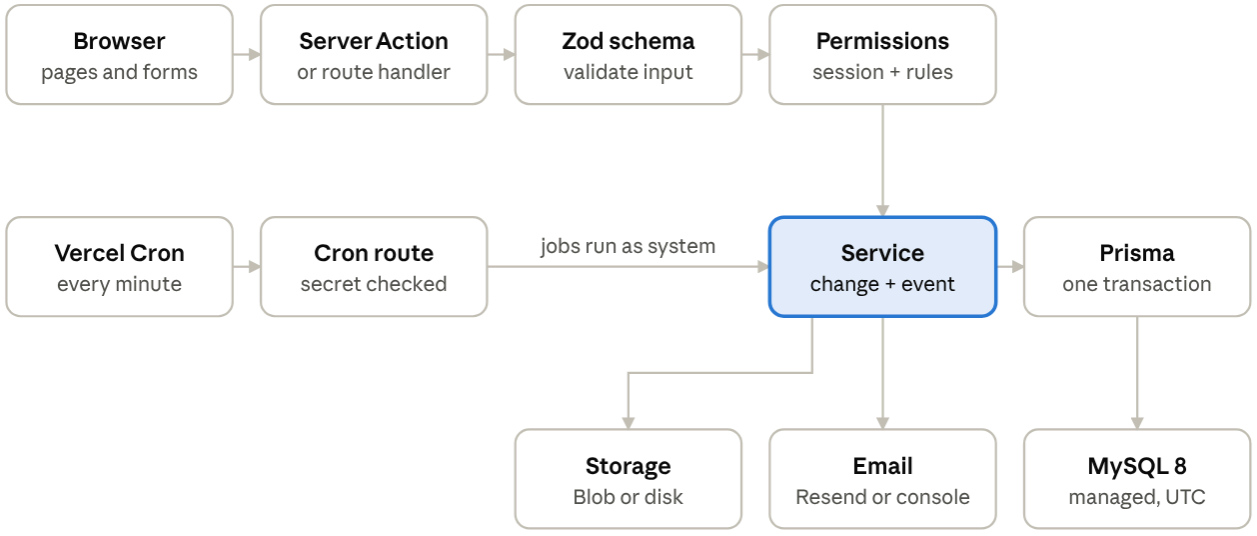
Layer	Choice	Notes
Database	MySQL 8 (managed; same region as Vercel functions)	`utf8mb4`, UTC everywhere; Docker Compose MySQL for local work
ORM	Prisma	Migrations with `prisma migrate`; `fullTextSearch` preview feature for MySQL FULLTEXT
Validation	Zod	One schema per action, shared by client form and server action; env vars validated at boot
Forms	react-hook-form + `@hookform/resolvers/zod`	Client-side feedback; server re-validates every time
Styling	Tailwind CSS	README tokens as CSS variables; `dark` class strategy plus "match system"
UI primitives	Radix UI (via shadcn/ui components, restyled)	Dialogs, menus, selects, popovers, tabs, tooltips with accessible focus handling
Fonts	Figtree 400–700, JetBrains Mono 500 via `next/font`	Mono for ids, timers and labels
Auth	Custom sessions: `Session` table, httpOnly secure cookie, argon2id hashes	Login rate limit; invite and reset tokens stored hashed with expiry
Jobs	Vercel Cron to `/api/cron/*` with `CRON_SECRET`	Local: a script that calls the same routes
Email	Resend (prod), console (dev) behind `lib/email.ts`	React Email templates for notifications and digest
Files	Vercel Blob (prod), local disk (dev) behind `lib/storage.ts`	Size and MIME limits enforced server-side
Dates	`date-fns` + `date-fns-tz`	Display in Africa/Accra (UTC+0, no daylight saving)
Drag and drop	`@dnd-kit`	Board view
Charts	Recharts	Reports
AI assist	Anthropic API, server-side only	Phase 9

Layer	Choice	Notes
Testing	Vitest (unit, services against a test MySQL), Playwright (end-to-end)	Permissions, SLA and forwarding must be covered
Quality	ESLint, Prettier, Husky + lint-staged, GitHub Actions CI	CI runs typecheck, lint, tests and `prisma validate`
Monitoring	Vercel logs + Sentry	Added before launch

Architecture and engineering rules

Every request passes through one path: UI → Zod validation → session and permission check → a domain service → Prisma in a transaction. The UI never talks to Prisma for writes, and nothing trusts the client.

Every write passes validation, permissions and one service transaction



request path · user writes and scheduled jobs share one service layer

User actions and cron jobs both end in the same service functions, so rules and event logging are written once.

Folder structure

```
app/
  (auth)/sign-in, set-password, forgot-password
  (app)/home, inbox, tickets/[number], sent, mentions, escalations,
        workload, departments/[slug], help, kb, assets, reports,
        admin/*, profile
  api/cron/*            Vercel Cron targets (SLA, escalate, autoclose,
```

```
digest, recurring)
  api/uploads, api/files/[id], api/inbound-email
components/
  ui/                    restyled Radix primitives
  tickets/, inbox/, layout/, ...
lib/
  db.ts                  single Prisma client
  env.ts                 Zod-validated environment
  auth/                  session, password, tokens, rate limit
  permissions.ts         the only place access rules live
  sla.ts                 pure deadline maths
  validation/            Zod schemas per feature
  services/              ticketService, userService, departmentService, ...
  storage.ts, email.ts   swappable adapters
  ids.ts                 MLT- / MLA- formatting and parsing
prisma/schema.prisma, prisma/seed.ts
tests/unit, tests/integration, tests/e2e
```

Rules every phase follows

1. **Permissions live in `lib/permissions.ts` and run on the server.** It exports per-item checks (`canView`, `canAssign`, `canForward`, …) and `visibleTicketsWhere(user)`, which returns a Prisma `where` clause. Every list, count, search and report builds on that clause, so confidential tickets cannot leak through a badge or search result.
2. **Every ticket change goes through a service function inside `prisma.$transaction`.** The function writes the change and its `TicketEvent` together. Events are append-only: never updated or deleted. They are also the audit trail.
3. **Forwarding** requires a reason, closes the current `TicketLeg` (`outAt`), opens a new leg in the target department, clears the assignee and writes a `forwarded` event, all in one transaction.
4. **SLA maths are pure functions in `lib/sla.ts`** that take `now` as an argument: `computeDueAt`, `percentElapsed`, `slaState` (ok, warning, breached). The UI, the cron jobs and the tests share them. Targets come from the `Setting` table.
5. **Validation:** each Server Action parses its input with a Zod schema from `lib/validation/` and returns a typed result (`{ ok: true, data }` or `{ ok: false, fieldErrors }`). Forms use the same schema on the client.
6. **Cron jobs are idempotent queries,** for example "breached tickets with no level-1 escalation". A missed or doubled run causes no harm. Each route checks `CRON_SECRET`.
7. **Admin changes are audited** in an `AuditLog` table (who, what, before, after).
8. **Copy is British English as in the prototype.** Both themes and `prefers-reduced-motion` are supported on every screen.

9. **Each phase:** plan, then schema and services with tests, then UI against the matching screenshot, then run, review and commit.

Data model

We implement the README's data model in Prisma, with the MySQL-specific choices below. Tables arrive in the phase that first needs them.

Model	Key fields	MySQL / Prisma notes	Phase
User	name, email (unique), passwordHash, role, departmentId, branch, phone, ext, signature, theme, density, notificationPrefs, active	Role and branch as enums; `active` to disable leavers without deleting history	1
Session, AuthToken	tokenHash, userId, expiresAt, type (invite or reset), usedAt	Store only SHA-256 hashes of tokens	1
Department	name, slug, hue, headUserId, emailAlias, confidentialByDefault	Colour derived from hue: `oklch(0.6 0.11 hue)`	1
ServiceType	departmentId, name, description, defaultPriority, needsApproval, fields (Json), attachmentRequired	Form field definitions are display-only, so JSON is fine	1
Ticket	number (auto-increment from 24827), subject, description, requesterId, departmentId, assigneeId, serviceTypeId, priority, status, channel, confidential, fields (Json), createdAt, dueAt, firstResponseAt, resolvedAt, csat	Indexes on (departmentId, status), (assigneeId, status), dueAt; FULLTEXT on subject + description; `DateTime(3)`	2
TicketLeg	ticketId, departmentId, inAt, outAt, byUserId, reason	One open leg per ticket (`outAt` null)	2
TicketEvent	ticketId, type (enum), byUserId, body (Text), meta (Json), mentions (Json), createdAt	Append-only; index (ticketId, createdAt)	2

Model	Key fields	MySQL / Prisma notes	Phase
Mention	eventId, userId or departmentId	A real table, because the Mentions page filters by it	3
Attachment	eventId, name, size, mime, storageKey	Files served through a permission-checked route	3
Escalation	ticketId, level, at, acknowledgedAt, acknowledgedById	Unique (ticketId, level) keeps cron idempotent	4
Notification	userId, ticketId, kind, readAt	Feeds the dropdown and digest	4
OutOfOffice	userId, until, delegateUserId		4
Setting	key, value (Json)	SLA targets, rule toggles, retention	4
SavedView	userId, name, filters (Json)		5
Task, TimeEntry, TicketLink	as README; links always written with their inverse	Ticket also gains parentId, mergedIntoId	6
Asset, Policy, Claim	Asset code `MLA-…` unique string; Policy is a stub table		6
Article, ArticleVote, Announcement	as README		7
Template, AuditLog	Template repeat enum, nextRunAt		8

Rule of thumb for JSON columns: if we will filter, sort, count or report on a value, it gets a real column or table. JSON is only for data we display as-is.

Implementation phases

Ten phases, each ending with something we can run, check against the prototype and commit. Phases 0–4 are the minimum launchable product: people can sign in, raise, assign, discuss, forward and resolve tickets with working deadlines and escalations.

Phase 0: Foundation

Goal: an empty, correctly configured project; no features.

- Next.js App Router project, TypeScript strict, ESLint, Prettier, Husky + lint-staged.
- Tailwind with all README tokens as CSS variables (light and dark), Figtree and JetBrains Mono via `next/font`, logos in `public/`.
- Prisma connected to MySQL 8; `docker-compose.yml` for local MySQL (plus a test database).
- `lib/env.ts` validating every variable with Zod; `.env.example`.
- Stubs for `lib/db.ts`, `lib/storage.ts`, `lib/email.ts`, `lib/permissions.ts`, `lib/sla.ts`.
- Vitest and Playwright configured; GitHub Actions CI (typecheck, lint, test, `prisma validate`).
- First Vercel deployment connected to the managed MySQL instance.

Check: `npm run dev` shows a blank page in Figtree; `npx prisma studio` connects; CI is green; the Vercel preview deploys.

Phase 1: Data model, seed, sign-in and app shell

Goal: the three roles can sign in and see their own navigation.

- Schema for User, Session, AuthToken, Department, ServiceType; migration.
- Seed: 12 departments with heads, members, admin Ama Serwaa, branches, service catalogue with form fields, all from the prototype; known dev password.
- Auth: sign-in, sign-out, session cookie, middleware protecting `(app)`, login rate limiting, forgot password and set-password pages with hashed single-use tokens.
- Admin "Add person" (minimal form) that sends the invite email.
- App shell: 240px navy sidebar with role-based nav, 58px header, off-canvas menu under 760px, dark mode toggle with "match system".
- `permissions.ts` with role rules and confidential-ticket rules, fully unit-tested.

Tests: permissions matrix (member, head, admin × own, other department, confidential); password hashing; token expiry and single use. **Check:** sign in as Kojo Appiah (head of IT), Esi Quaye (member) and Ama Serwaa (admin); the sidebar differs per role; a reset link works end to end with console email.

Phase 2: Create a ticket and the department inbox

Goal: a member raises a request and the receiving head triages it.

- Schema for Ticket, TicketLeg, TicketEvent; ticket counter starting at 24827; `lib/ids.ts`.

- `ticketService.create` (ticket + first leg + `created` event + dueAt from `lib/sla.ts`), `assign`, `setPriority` (recomputes dueAt from createdAt), `setStatus`.
- New ticket wizard: step 1 description with keyword-matched department and category suggestions; step 2 details with service-specific required fields, priority, confidential toggle (defaulted per department); draft restored on reopen.
- Inbox list: tabs (Open, Unassigned, Assigned to me, Resolved), priority filter, "Due within 4h" chip, inline priority, assignee and status edits, bulk bar for heads, admin department filter.
- Sent page.

Tests: dueAt for each priority; priority change recomputes from createdAt; members cannot assign; confidential ticket hidden from another department's member in list and count queries. **Check:** a member sends an IT request; the IT head sees it under Unassigned, assigns it and changes priority.

Phase 3: Ticket detail, conversation and forwarding

Goal: the full working life of a ticket on one page.

- Ticket page: header with tags, Journey stepper from TicketLeg, deadline ring, custom fields grid.
- Conversation: replies and internal notes, grouping within 15 minutes, system events as chips, @mentions of people and departments (Mention table), signature, canned replies, draft autosave.
- Attachments through `lib/storage.ts`, with image and PDF preview, size and type limits, and a permission-checked download route.
- Forward with required reason (closes leg, opens leg, clears assignee); resolve; reopen; rating with 1–2 star feedback dialog that creates a head follow-up.
- Undo for 6 seconds after send, resolve and forward (delayed commit on the client, or a compensating event).
- `firstResponseAt` set on the first reply from the handling department.

Tests: forwarding closes and opens legs correctly; history survives multiple forwards; internal notes never visible to the requester; mentioned member gains view access. **Check:** IT forwards a printer ticket to Facilities; the Journey shows both departments with time spent in each.

Phase 4: Deadlines, escalation and notifications

Goal: the system acts on time without anyone watching.

- Cron routes (every minute): SLA warning at 75%, breach, level-1 escalation to head, level-2 to admin/COO after a further window; auto-close resolved tickets after 3 days; out-of-office reassignment.
- Escalations page with Acknowledge.
- Notification table, header dropdown with unread dot and "mark all read".
- Email notifications via Resend and the 07:30 daily digest.
- Settings: editable SLA targets and rule toggles (admin).

Tests: `slaState` boundaries; escalation job is idempotent (two runs, one escalation); auto-close skips rated or reopened tickets. **Check:** set Urgent to 2 minutes, create an urgent ticket, watch it turn amber, breach and escalate.

Phase 5: Home, Board, Calendar, Workload, Departments

- Home per role (stat cards, Needs you, Due in 24h, Your requests, Recent updates, low ratings for heads, announcements placeholder).
- Board view with drag-and-drop status changes; Calendar of due dates; split view at 1180px and above; saved views; hover previews.
- Workload page for heads; Departments directory and department pages; Mentions page; global search with FULLTEXT.

Check: every Home card links to the right filtered list; dragging a card changes status and writes an event.

Phase 6: Tasks, time, links, merge, assets, policy lookup

- Ticket tasks with progress; time tracking (timer, +15m, +30m, +1h).
- Ticket links (blocked by, blocks, related to) with automatic inverse and a Blocked tag.
- Merge and sub-tickets, with duplicate detection on create.
- Asset register (`MLA-` codes) with ticket history and "Report a fault"; policy and claim lookup from a stub table.

Check: merging moves the conversation reference and both tickets show `merged` / `mergedin` events.

Phase 7: Knowledge base, reports, announcements, profile

- Help Center and knowledge base with helpful votes and suggestions in the wizard.
- Reports: KPIs, volume and SLA charts, leaderboard, satisfaction, forwarding flows, CSV export, scheduled email report.

- Announcements banner and posting; Profile and settings (photo, phone, extension, signature, theme, density, notification preferences, out of office); print/PDF view of a ticket.

Check: report numbers match a hand count on seed data; heads see only their department's figures.

Phase 8: Administration, audit and compliance

- Admin tabs: departments and heads, people and roles (create, disable, change role), SLA and escalation, forwarding and mention rules, templates and recurring tickets (cron), channels (department email aliases, inbound email-to-ticket webhook), confidentiality defaults and retention jobs, audit log with filter and CSV export.
- Every admin change writes an AuditLog entry.

Check: disabling a user ends their sessions immediately; an inbound email creates a ticket in the right department.

Phase 9: AI assist, polish and launch

- AI assist (summarise, suggest reply, routing check) via the Anthropic API, server-side, with "check before using" copy and no confidential content sent without a rule allowing it.
- Motion spec with `prefers-reduced-motion`, skeleton loaders, empty states, first-run tour per role.
- Accessibility pass (keyboard, focus, AA contrast in both themes).
- Playwright flows: create → assign → forward → resolve → rate; security review; Sentry; backups confirmed; replace seed data with real miLife data.

Check: all end-to-end tests pass on the production-like preview; a pilot department uses it for one week.

Risks, open questions and launch checklist

Risk	Effect	Mitigation
Serverless functions exhaust MySQL connections	Errors under load	One Prisma client, low `connection_limit`, provider pooling; load-test before launch
Vercel Hobby plan only runs crons daily	No SLA warnings or escalations	Vercel Pro before Phase 4

Risk	Effect	Mitigation
Confidential data leaks through lists, counts or search	Privacy breach	`visibleTicketsWhere` used by every query; dedicated tests
Scope creep from the 17 screens	Late launch	Launch after Phase 4 with a pilot department; later phases ship incrementally
Database outside Ghana	Data Protection Act 2012 (Act 843) concern	Confirm with Compliance before production data is loaded

Open questions

- [ ] Which managed MySQL provider and region?
- [ ] Sending domain for Resend (for example `servicedesk.milife.com.gh`)?
- [ ] Who is the COO recipient for level-2 escalations, and how long after breach?
- [ ] Is the company email domain fixed, so invites can be restricted to it?
- [ ] Which department pilots first?

Launch checklist

- [ ] Real departments, heads, people and service catalogue replace seed data
- [ ] Production env vars set and validated; `CRON_SECRET` rotated
- [ ] Database backups and restore tested
- [ ] Sentry receiving errors
- [ ] Playwright suite green on the production preview
- [ ] Compliance sign-off on hosting region and retention periods